

Java Backend Architecture

Interview Series

Chapter 10.6

Autocomplete & Fuzzy Search

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 10.6 – Autocomplete & Fuzzy Search

Introduction

Autocomplete (search-as-you-type) and fuzzy search are two foundational features of modern search UIs. Autocomplete reduces user effort by surfacing suggestions as the user types; fuzzy search recovers gracefully from typos and spelling variations. Elasticsearch provides four distinct mechanisms for autocomplete — edge n-gram, match_phrase_prefix, completion suggester, and the search_as_you_type field type — each with different trade-offs in speed, flexibility, and index complexity.

Fuzzy search uses Levenshtein edit distance to find terms that differ from the query by N character operations (insertions, deletions, substitutions, transpositions). The AUTO fuzziness setting adapts the allowed edit distance to query length, balancing recall against precision.

Autocomplete Approaches

Autocomplete Approaches in Elasticsearch			
Edge N-gram	match_phrase_prefix	Completion Suggester	Search-as-you-type
Index-time tokenization	Query-time expand	Dedicated data struct	Special field type
index: edge_ngram	No special mapping	In-memory FST	Auto ngram+prefix
search: standard	Expands last token	Extremely fast	Shingles support
Fast, flexible	Good for phrase	Fuzzy option	Easy setup
Good for prefix match	Slower at scale	Payload support	ES 7.2+ only
Rule of thumb: completion suggester for speed, edge n-gram for flexibility			

Levenshtein Edit Distance

Levenshtein Edit Distance Examples			
Source	Target	Distance	Operation
kitten	sitten	1	substitution: k->s
sitten	sittin	1	substitution: e->i
sittin	sitting	1	insertion: +g
color	colour	1	insertion: +u
algorithm	algorithmm	1	insertion: +m
helo	hello	1	insertion: +l
AUTO fuzziness: 0 for len<3, 1 for len 3-5, 2 for len>5			

N-gram vs Edge N-gram

N-gram vs Edge N-gram Tokenization

Input: "quick"

n-gram (min=2, max=3)

qu, ui, ic, ck, qui, uic, ick
(all substrings of length 2-3)
Use for infix/suffix search

edge n-gram (min=2, max=5)

qu, qui, quic, quick
(prefixes only)
Use for prefix/autocomplete

edge n-gram produces fewer tokens -> smaller index, faster queries

Set `search_analyzer=standard` to avoid double-ngram at query time

Key Definitions

autocomplete	Surfacing suggestions as a user types, based on prefix/partial match
edge n-gram	Tokenizer producing prefixes of each token (qu, qui, quic, quick)
n-gram	Tokenizer producing all substrings of configurable length
completion suggester	In-memory FST-based structure for extremely fast autocomplete
search_as_you_type	ES 7.2+ field type that auto-creates ngram and prefix sub-fields
match_phrase_prefix	Expands the last token of the query string as a prefix at query time
fuzzy query	Finds terms within N Levenshtein edit distance from the query term
fuzziness	Number of allowed edits; AUTO = 0 for len<3, 1 for 3-5, 2 for >5
Levenshtein distance	Min single-char edits (insert/delete/substitute) to transform one string to another
Damerau-Levenshtein	Extension adding transpositions (ab->ba = 1 edit); used by ES by default
prefix_length	Number of leading characters that must match exactly before fuzzy applies
max_expansions	Max number of fuzzy variants to expand; higher = better recall, more CPU
transpositions	Adjacent character swaps (teh->the) counted as 1 edit (Damerau extension)
FST	Finite State Transducer — memory-efficient automaton used by completion suggester
suggest mode	Controls when completion suggestions are offered: missing, popular, always

Approach Comparison

Approach	Speed	Index Size	Flexibility	Fuzzy?	Best For
Edge n-gram	Fast	Medium+	High	No (combine with fuzzy)	Product names, rich filtering
match_phrase_prefix	Medium	Normal	Medium	No	Simple phrase autocomplete

Completion suggestions	Very fast	Low (FST)	Low	Yes (built-in)	Search box dropdown, global nav
search_as_you_type	Fast	Medium	Medium	No	General-purpose, ES 7.2+

Code Examples

1. Edge N-gram autocomplete mapping

```
// PUT /products { "settings": { "analysis": { "analyzer": { "autocomplete_index": {
"tokenizer": "standard", "filter": ["lowercase", "edge_ngram_filter"] },
"autocomplete_search": { "tokenizer": "standard", "filter": ["lowercase"] } }, "filter": {
"edge_ngram_filter": { "type": "edge_ngram", "min_gram": 2, "max_gram": 20 } } }, "mappings":
{ "properties": { "name": { "type": "text", "analyzer": "autocomplete_index", // edge-ngram at
index time "search_analyzer": "autocomplete_search" // standard at query time! } } } } //
Query: simply use match { "query": { "match": { "name": { "query": "wirel", "analyzer":
"autocomplete_search" } } } } // Matches: "wireless", "wireless headphones", etc.
```

2. match_phrase_prefix for phrase autocomplete

```
{ "query": { "match_phrase_prefix": { "name": { "query": "running sh", // expands "sh" ->
shoes, shorts, shirt... "max_expansions": 50, // limit prefix expansions for performance
"slop": 1 // allow 1 word between terms } } } } // Returns: "running shoes", "running shorts",
"running shirt" // Note: only the LAST token is prefix-expanded // Better alternative for
production: use search_as_you_type field type
```

3. Completion Suggester mapping and query

```
// Mapping: { "mappings": { "properties": { "suggest": { "type": "completion", "analyzer":
"simple", "search_analyzer": "simple", "preserve_separators": true,
"preserve_position_increments": true, "max_input_length": 50 }, "name": { "type": "text" } } }
} // Indexing a document with suggest: { "name": "Apple iPhone 15 Pro", "suggest": { "input":
["iPhone", "iPhone 15", "iPhone 15 Pro", "Apple iPhone"], "weight": 10 // higher weight =
ranked higher in suggestions } } // Query: // POST /products/_search { "suggest": {
"product_suggest": { "prefix": "ipho", "completion": { "field": "suggest", "size": 5, "fuzzy":
{ "fuzziness": 1, "unicode_aware": true }, "contexts": { "category": ["electronics"] } } } } }
```

4. search_as_you_type field type (ES 7.2+)

```
// Mapping: { "mappings": { "properties": { "name": { "type": "search_as_you_type",
"max_shingle_size": 3, // creates 2- and 3-word shingles "analyzer": "standard" } } } } // ES
creates sub-fields automatically: // name (2-gram) // name._2gram, name._3gram (shingle fields)
// name._index_prefix (edge_ngram of last shingle) // Query – target all sub-fields: { "query":
{ "multi_match": { "query": "wireless head", "type": "bool_prefix", "fields": ["name",
"name._2gram", "name._3gram", "name._index_prefix"] } } } // Java:
SearchResponse<ProductDocument> resp = esClient.search(s -> s.index("products").query(q ->
q.multiMatch(m -> m.query("wireless head").type(TextQueryType.BoolPrefix).fields("name",
"name._2gram", "name._3gram", "name._index_prefix"))), ProductDocument.class );
```

5. Fuzzy search with match query

```
{ "query": { "match": { "name": { "query": "wiereless headfonez", // typos! "fuzziness":
"AUTO", "prefix_length": 2, // first 2 chars must match exactly "max_expansions": 50, // limit
fuzzy variants "fuzzy_transpositions": true // ab->ba counts as 1 edit } } } } // For specific
```

```
control: // fuzziness: 0 (exact), 1 (1 edit), 2 (2 edits) // AUTO = 0 for length < 3, 1 for
3-5, 2 for > 5 // Standalone fuzzy query (term-level, not analyzed): { "query": { "fuzzy": {
"category.keyword": { "value": "elecronics", // missing 't' "fuzziness": 1, "prefix_length": 3
} } } }
```

6. Phrase suggerter (did-you-mean)

```
{ "suggest": { "text": "wireles headfones bluetooth", "phrase_suggestion": { "phrase": {
"field": "name", "size": 3, "gram_size": 3, "direct_generator": [{ "field": "name",
"suggest_mode": "always", "min_word_length": 2 }], "highlight": { "pre_tag": "<em>",
"post_tag": "</em>" }, "collate": { "query": { "source": { "match": { "{{field_name}}":
"{{suggestion}}" } } }, "params": { "field_name": "name" }, "prune": true // only return
suggestions that actually have results } } } } } // Response suggestions: "wireless headphones
bluetooth"
```

7. Java: Spring autocomplete endpoint

```
@RestController @RequestMapping("/api/search") public class AutocompleteController { private
final ElasticsearchClient esClient; @GetMapping("/suggest") public List<String>
suggest(@RequestParam String prefix, @RequestParam(defaultValue = "10") int size) throws
IOException { // Approach 1: Completion suggerter SearchResponse<ProductDocument> response =
esClient.search(s -> s .index("products") .suggest(sg -> sg .suggesters("product_suggest", sug
-> sug .prefix(prefix) .completion(c -> c .field("suggest") .size(size) .fuzzy(f ->
f.fuzziness("1")) ) ) ), ProductDocument.class ); return response.suggest()
.get("product_suggest") .stream() .flatMap(s2 -> s2.completion().options().stream()) .map(opt
-> opt.text()) .collect(Collectors.toList()); } @GetMapping("/autocomplete") public
List<String> autocomplete(@RequestParam String query, @RequestParam(defaultValue = "10") int
size) throws IOException { // Approach 2: search_as_you_type SearchResponse<ProductDocument>
response = esClient.search(s -> s .index("products") .size(size) .query(q -> q.multiMatch(m ->
m .query(query) .type(TextQueryType.BoolPrefix) .fields("name", "name._2gram", "name._3gram")
)), ProductDocument.class ); return response.hits().hits().stream() .map(h ->
h.source().getName()) .collect(Collectors.toList()); } }
```

Interview Questions & Answers

Q1. What are the four main autocomplete approaches in Elasticsearch?

1) Edge n-gram: tokenize at index time with edge n-gram filter, search with standard. Flexible, supports filters/aggregations, but increases index size. 2) match_phrase_prefix: expands last query token as a prefix at query time. Simple, no special mapping, but slower at scale and limited to prefix of last word. 3) Completion suggerter: dedicated in-memory FST structure, extremely fast (sub-ms), with built-in fuzzy support. Limited to prefix matching, no post-filtering. 4) search_as_you_type: special field type (ES 7.2+) auto-creating ngram+shingle sub-fields.

Q2. What is the difference between n-gram and edge n-gram tokenizers?

n-gram produces ALL substrings of configured length. Input='quick', min=2, max=3: qu, ui, ic, ck, qui, uic, ick. Useful for infix/substring search (finding 'ui' inside 'quick'). edge n-gram produces only PREFIX substrings: qu, qui, quic, quick. Much smaller token count — better for autocomplete. Choose edge n-gram for prefix autocomplete; choose n-gram only if you need substring matching.

Q3. Why must you set a different search_analyzer when using edge n-gram?

If you use the edge n-gram analyzer for both indexing AND searching, the query 'quick' would be tokenized to [qu, qui, quic, quick] at query time, and each ngram would match many documents unexpectedly. You want the query to match documents that contain tokens with the query as a prefix — so use ``standard`` (or ``simple``) as `search_analyzer` to keep the query as a single token, matched against the ngram-indexed field.

Q4. How does the Completion Suggester work internally?

The completion suggester builds a Finite State Transducer (FST) in memory at shard startup. The FST maps input strings to their associated document IDs and weights. Prefix lookup traverses the FST in $O(\text{prefix_length})$ time — effectively constant. This makes it extremely fast for simple prefix lookups. Trade-off: the FST is in JVM heap; large suggest fields increase heap pressure. Doesn't support post-filtering by arbitrary query conditions (only context filters).

Q5. What are context filters in the Completion Suggester?

Context filters let you limit completions to specific categories or geo regions. Define contexts in the mapping: ``category`` context of type 'category', or ``location`` of type 'geo'. When indexing, provide context values per document. At query time, specify which context values to filter by. Example: user in 'electronics' category sees only electronics product suggestions. This avoids returning cross-category results in a context-aware autocomplete.

Q6. Explain the search_as_you_type field type.

Introduced in ES 7.2, ``search_as_you_type`` creates 4 sub-fields automatically: the root field (2-gram shingles), ``._2gram`` (2-word shingles), ``._3gram`` (3-word shingles), and ``._index_prefix`` (edge n-gram of the last shingle). Query with `multi_match` type=`bool_prefix` across all sub-fields. This handles both single-token prefix ('wir') and multi-token phrase prefix ('wireless head') efficiently without manual mapping.

Q7. What is fuzziness in ES and how does AUTO work?

Fuzziness specifies the maximum allowed Levenshtein edit distance between the query term and indexed terms. AUTO dynamically sets the limit based on query term length: 0 edits for strings < 3 chars (too short to meaningfully fuzzy-match), 1 edit for 3-5 chars (moderate tolerance), 2 edits for > 5 chars (allows catching most typos). You can also set `AUTO:3,6` to customize the thresholds.

Q8. What is prefix_length in fuzzy queries and why is it important?

``prefix_length`` specifies how many leading characters must match exactly before fuzzy expansion applies. Default is 0. Setting `prefix_length=2` means the first 2 characters must be identical — ES only expands fuzzy variants for terms that share the same prefix. Benefits: (1) prevents too many false positives (very different words sharing edit distance 1), (2) dramatically reduces `max_expansions` computation since few terms share prefixes with the query beyond the `prefix_length`.

Q9. What is max_expansions in fuzzy queries?

When ES computes fuzzy variants, it expands the query term into all indexed terms within the edit distance. ``max_expansions`` (default 50) caps how many expanded terms are used. Higher `max_expansions` = better recall but more CPU (each expansion is a separate term lookup). Too low can miss valid fuzzy matches. In practice, 50-100 is a good balance. Note: this is a per-shard limit, so results can vary between shards.

Q10. How do transpositions affect fuzzy matching?

Standard Levenshtein counts a transposition (swapping adjacent chars: 'teh' -> 'the') as 2 operations (delete 'e', insert 'e'). Damerau-Levenshtein (which ES uses by default when ``fuzzy_transpositions: true``) counts it as 1. This makes 'teh' -> 'the' match with `fuzziness=1`, which is desirable since transpositions are very common typos. Set ``fuzzy_transpositions: false`` only if you need strict Levenshtein semantics.

Q11. What is the phrase suggester and how does `collate` improve it?

The phrase suggester generates 'did you mean' corrections for an entire phrase by checking all term combinations for nearby terms in the index. Without ``collate``, it might suggest a corrected phrase that has zero results in the index. ``collate`` runs a verification query for each suggestion and only returns suggestions that actually have matching documents (when ``prune: true``). This prevents 'did you mean X' when X also returns no results.

Q12. How do you implement typo-tolerant search in a Java Spring Boot service?

Add ``fuzziness: AUTO`` to your match query. Use ``prefix_length: 2`` to prevent false positives. Combine with `multi_match` for coverage across name/description fields. In the Java client: ``match(m -> m.field('name').query(input).fuzziness('AUTO').prefixLength(2).maxExpansions(50))``. For production, A/B test fuzzy vs. exact to verify it improves CTR without degrading precision. Log zero-result queries and enable fuzzy only as a fallback when exact gives zero results.

Q13. What is the wildcard query and when should you avoid it?

The ``wildcard`` query matches terms using `*` (zero or more chars) and `?` (single char): ``{`name.keyword': 'wire*}``. Leading wildcards (`*less`) force a full scan of the inverted index — extremely expensive. Never use leading wildcards in production at scale. Safe usage: trailing wildcards (`'wire*'`) with a ``prefix_length`` equivalent; or use the ``wildcard`` field type for patterns that need to support wildcards. Prefer edge n-gram or prefix query over wildcard for autocomplete.

Q14. What is the regexp query and what are its performance implications?

The ``regexp`` query applies a regular expression to terms in the inverted index. It iterates over all terms that match the regex — $O(\text{terms in index})$. Complex regex with backtracking can be extremely slow. Mitigations: set ``flags`` to limit regex features, use ``max_determinized_states`` to cap automaton complexity, always combine with a filter that drastically narrows the doc set first. For most use cases, ngram-based approaches are preferable.

Q15. How do you handle multi-word autocomplete ('running sho...') vs single-word?

Single-word prefix is handled by edge n-gram or prefix query on the last token. Multi-word phrase prefix is handled by: (1) ``match_phrase_prefix`` — expands the last word as a prefix, requires previous words to match in order. (2) ``search_as_you_type`` field type with shingle sub-fields — handles both single and multi-word naturally. (3) Completion suggester with multi-input array — manually populate with all expected phrase prefixes at index time.

Q16. What is the `suggest\_mode` in term/phrase suggesters?

``suggest_mode`` controls when suggestions are generated: ``missing`` — only suggest when the query term doesn't appear in the index (conservative, avoids unnecessary suggestions). ``popular`` — suggest only terms that are more popular than the original query term. ``always`` — always generate suggestions regardless. Use ``missing`` for did-you-mean (only fires when term not found); use ``always`` for exploration UIs.

Q17. How would you architect a high-traffic autocomplete service?

1) Use completion suggester (in-memory FST, sub-ms latency). 2) Cache frequent query prefixes in Redis with a short TTL (1-5 min). 3) Debounce autocomplete requests client-side (trigger after 300ms pause). 4) Use a dedicated autocomplete index separate from the main search index (different mappings, retention, and resource allocation). 5) Precompute and store popular queries in the suggester index. 6) Implement rate limiting to prevent suggestion scraping.

Q18. What are context-aware completions and give a real-world example?

Context-aware completions restrict suggestions to documents matching a specific context field defined at mapping time. Example: an e-commerce site where users are browsing the 'Electronics' department — the autocomplete should only suggest electronics product names. Map a ``category`` context on the completion

field, set `category='electronics'` on electronics documents, and pass ``contexts: {category: ['electronics']}`` in the suggest query. Without context, 'wire' might suggest 'wire transfer' (banking) in addition to 'wireless headphones'.

Q19. How do you handle real-time updates to completion suggester data?

The completion suggester FST is rebuilt per segment, like any other indexed field. Each new document/update is available in the suggester after the next segment refresh (default every 1 second). There's no separate build step needed. However, the in-memory FST grows with new documents — manage heap by: (1) periodically force-merging to reduce segment count (and thus FST count per shard), (2) monitoring JVM heap, (3) setting ``max_input_length`` to cap very long inputs.

Q20. What is the difference between prefix query and match_phrase_prefix?

``prefix`` query matches documents where the field contains a term that starts with the given prefix. It's a term-level query — NOT analyzed. Use on ``keyword`` fields or with the last token from a pre-tokenized field. ``match_phrase_prefix`` accepts a full phrase, analyzes it, and treats the last token as a prefix. It also requires the preceding words to appear as a phrase (ordered). For single-token prefix, they behave similarly; for multi-word autocomplete, `match_phrase_prefix` is more powerful.

Q21. How would you combine autocomplete with filtering (e.g., 'in-stock, electronics')?

Completion suggester: use context filters (category context with 'electronics'). Cannot easily combine with arbitrary filters. Edge n-gram approach: use ``match`` query for the partial text PLUS ``filter`` clauses in a ``bool`` query for structured filters — this is the most flexible option. Example: `bool` with `must=[match on name with ngram]`, `filter=[term(category, electronics), term(in_stock, true)]`. This is why edge n-gram is preferred for production search UIs that need faceted autocomplete.